

과제 7 · PLAN · DO · SEE DIARY 2

검증 기록 및 제출 자료

검증 안내서

어디로 가나요

공개 주소: <https://07end.vercel.app/>

무엇을 하나요

1. 새 계정을 만든 뒤 로그인·로그아웃하고, 로그인하지 않은 상태에서는 다이어리 자료가 보이지 않는지 확인합니다.
2. 비밀번호 저장 방식과 로그인 세션을 확인하고, 로그아웃 뒤 같은 자료 요청이 거절되는지 확인합니다.
3. 두 계정의 자료가 서로 섞이지 않는지 확인한 뒤 5 일 기록·규칙 변경·합계/평균·내보내기·계정 삭제까지 점검합니다.

무엇이 보이면 통과인가요

가입·로그인·로그아웃이 동작하고 비로그인 상태에서는 자료 대신 로그인 화면이 보여야 합니다. 비밀번호 원문과 인증 비밀번호가 노출되지 않고, 로그인 세션이 끊긴 뒤에는 이전 접근이 거절되며, 다른 계정의 자료는 읽기·수정·삭제·목록에서 섞이지 않아야 합니다. 5 일 기록과 규칙 변경 전후 비교, 합계·평균 대조, 내보내기와 계정 삭제까지 카드 기준과 일치하면 통과입니다.

안 될 때

로그인 상태와 현재 계정을 먼저 확인합니다. 인증이 예상과 다르면 로그아웃 후 같은 주소를 다시 열어 로그인 화면으로 돌아오는지 확인하고, 접근 차단 검증은 요청·응답과 자료 건수를 다시 대조합니다. 제출용 기록에는 비밀번호·세션값·비밀키 원문을 남기지 않습니다.

AI 3 줄

AI 에게 맡긴 일 인증 코드 구조와 카드별 통과 기준을 대조하고, 필요한 검증 화면과 제출 문서의 구성·표·설명 정리를 맡겼다.

내가 판단한 일 실제 사용할 계정과 다이어리 자료, 5 일 기록의 질문·지표·계획 규칙, 규칙 변경 시점과 이유, 제출에 사용할 실제 캡처를 직접 확인하고 선택했다.

AI 말을 안 들은 일 카드에 없는 검증을 임의로 추가하거나 실제로 확인하지 않은 항목을 PASS 로 적지 않고, 과제 카드의 요구와 실제 동작을 기준으로 제출 자료를 구성했다.

1. 인증 방식 선택 · 가입 · 로그인

검증 항목	확인 내용	결과
인증 방식	직접 구현 + bcrypt ^6.0.0	PASS
선택 이유	인증 흐름을 직접 설명하고 서버 세션을 즉시 끊기 위해 선택	PASS
미선택 방법	Supabase Auth · JWT — 외부 의존과 토큰 무효화 관리 부담	PASS
회원가입	새 계정 생성 · 중복 아이디 가입 거절	PASS
로그인·로그아웃	로그인 후 다이어리 진입 · 로그아웃 후 로그인 화면	PASS
비로그인 접근	직접 접근 시 자료 대신 로그인 화면	PASS
로그인 실패 안내	없는 아이디 / 틀린 비밀번호 문구 동일	PASS

가입·로그인·로그아웃·세션·자료 소유권 검사는 api/pds.js 에서 직접 구현하고, 비밀번호 해시는 bcrypt(^6.0.0)에 맡겼습니다. Supabase Auth 와 JWT 도 검토했지만 인증 흐름을 직접 설명하고 로그아웃 즉시 세션을 끊기 위해 HttpOnly 쿠키 기반 서버 세션을 선택했습니다.

PLAN · DO · SEE DIARY 2

내 다이어리 로그인

내 계획과 기록은 로그인 후에만 보입니다.
처음 사용하는 경우 회원가입한 뒤 바로 다이어리를 열 수 있습니다.

아이디

비밀번호

로그인

회원가입

비밀번호 원문은 데이터베이스에 저장하지 않습니다. 로그인 세션은 HttpOnly 쿠키로 유지됩니다.

비로그인 직접 접근

PLAN · DO · SEE DIARY 2

회원가입

내 계획과 기록은 로그인 후에만 보입니다.
처음 사용하는 경우 회원가입한 뒤 바로 다이어리를 열 수 있습니다.

아이디

비밀번호

비밀번호 확인

회원가입

취소

비밀번호 원문은 데이터베이스에 저장하지 않습니다. 로그인 세션은 HttpOnly 쿠키로 유지됩니다.

회원가입

로그인

회원가입

회원가입이 완료되었습니다. 로그인해주세요.

회원가입 완료

회원가입

취소

이미 사용 중인 아이디입니다.

중복 가입 방지

로그인

회원가입

로그아웃되었습니다. 같은 자료 추소를 다시 열어도 로그인 전에는 기록이 보이지 않습니다.

로그아웃

로그인

회원가입

아이디 또는 비밀번호가 올바르지 않습니다.

틀린 비밀번호

로그인

회원가입

아이디 또는 비밀번호가 올바르지 않습니다.

없는 아이디

2. 비밀번호를 어떻게 맡아 두는지

검증 항목	확인 내용	결과
비밀번호 저장 방식	bcrypt 사용, package.json 기준 ^6.0.0	PASS
해시 생성 위치	api/pds.js 에서 bcrypt.hash(password, 12) 사용	PASS
DB 저장값	users.password_hash 에 원문이 아닌 bcrypt 해시 저장	PASS
같은 비밀번호 비교	같은 비밀번호 계정들의 password_hash 값이 서로 다름	PASS
로그인 검증	bcrypt.compare 로 입력 비밀번호와 저장 해시 비교	PASS
원문 노출 확인	로그인 응답에 비밀번호 원문과 해시값 없음	PASS

비밀번호는 원문으로 저장하지 않고 bcrypt 해시로 저장했습니다. 같은 비밀번호로 만든 계정들도 password_hash 값이 서로 다르게 저장되며, 로그인 응답에는 비밀번호 원문과 해시값이 포함되지 않습니다.

```
const userId=`user-${crypto.randomUUID()}`;
const passwordHash=await bcrypt.hash(password,12);
await client.query('INSERT INTO users(user_id,username,password_hash) VALUES($1,$2,$3)',[userId,username,passwordHash]);
```

bcrypt.hash(password, 12)로 비밀번호 해시 생성

```
if(action==='auth-login' && req.method==='POST'){
  const username=normalizeUsername(body.username), password=String(body.password||'');
  const q=await client.query('SELECT user_id,username,password_hash FROM users WHERE username=$1',[username]);
  const valid=q.rowCount?await bcrypt.compare(password,q.rows[0].password_hash):false;
  if(!valid)return bad(res,'아이디 또는 비밀번호가 올바르지 않습니다.',401,'INVALID_CREDENTIALS');
```

bcrypt.compare 로 로그인 입력값과 저장 해시 비교

1	SELECT username, password_hash
2	FROM users
3	LIMIT 5;

username	password_hash
text	text
rayeon	\$2b\$12\$SPKR1qfW2NIExy1H2wU5teU0bVQKYqKzNPHqIkzChuLt/Vj7mV9V6
test	\$2b\$12\$hVoprcQtScNMWN/NTPRqfeWPV.N1tzT1D3mPG8Ln9Pk7G.6BNivjy
rayeondary	\$2b\$12\$ahSB2yzc0SIhmwPWv2/.FuOk4RcU9WkzAEYt/exiMy9fK/CmFE1SK
admin	\$2b\$12\$/MvQwUR90wegfmQYueVxEeWoInL91J8y8y2wZsCCS8gDirA0uX0Im
rayeonrecode	\$2b\$12\$Mz088jFzXz1WmzRFFqxT9.kzayX9fDSxxIUHSZM6wLGHnsixsvwP.

DB users.password_hash 확인 — 같은 비밀번호도 서로 다른 bcrypt 해시로 저장

Name	×	Headers	Payload	Preview	Response	Initiator	Timing	Cookies
pds?action=auth-login	1				{ "ok": true, "user": { "username": "rayeon" }, "sessionExpiresInSeconds": 21600 }			
pds?action=bootstrap								

로그인 응답에 비밀번호 원문과 해시값 없음

3. 들어온 사람을 어떻게 기억하는지

검증 항목	확인 내용	결과
기억 방식	t07_session HttpOnly 쿠키 + auth_sessions 서버 세션	PASS
세션 생성	로그인 시 세션 해시와 expires_at 저장	PASS
세션 만료	Max-Age 21600 초(6 시간) 적용	PASS
쿠키 보호	HttpOnly · Secure · SameSite=Strict 적용	PASS
로그인 상태	인증된 자료 요청은 200 으로 정상 처리	PASS
로그아웃	세션 삭제 후 같은 자료 요청은 401 거절	PASS

로그인 상태는 URL 이나 JWT 가 아니라 HttpOnly 쿠키 기반 서버 세션으로 유지합니다. 세션 원문은 DB 에 그대로 저장하지 않고 해시로 저장하며, 6 시간 만료 기준을 두고 로그아웃 시 서버 세션과 쿠키를 함께 제거합니다.

인증값은 URL 에 넣지 않으며, 비밀번호 변경 시 기존 세션을 삭제해 이전 인증값을 사용할 수 없게 했습니다.

```
async function createAuthSession(client,res,userId){
  const raw=crypto.randomBytes(32).toString('base64url');
  const hash=sessionHash(raw);
  await client.query(`INSERT INTO auth_sessions(session_hash,user_id,expires_at) VALUES($1,$2,NOW()+($3||' seconds')::interval)`,[hash,userId,String(SESSION_MAX_AGE_SECONDS)]);
  setAuthCookie(res,raw);
}
```

로그인 시 auth_sessions 에 세션 해시와 만료 시간 저장

```
function setAuthCookie(res, token){
  res.setHeader('Set-Cookie', `${AUTH_COOKIE}=${encodeURIComponent(token)}; Path=/; HttpOnly; Secure; SameSite=Strict; Max-Age=${SESSION_MAX_AGE_SECONDS}`);
}
```

t07_session 쿠키에 HttpOnly · Secure · SameSite=Strict · Max-Age 적용

```
async function logoutSession(client,req,res){
  const raw=cookiesOf(req)[AUTH_COOKIE];
  if(raw)await client.query('DELETE FROM auth_sessions WHERE session_hash=$1',[sessionHash(raw)]);
  clearAuthCookie(res);
}
```

로그아웃 시 auth_sessions 세션 삭제 후 인증 쿠키 제거

Name	V...	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	P...	Cr...	Priori...
t07_session	p...	07start.verc...	/	2026-09-01T07:55:37.109Z	54	✓	✓	Strict			Medi...

Application 에서 t07_session 쿠키 보호 속성 확인

Request URL	Request Method	Status Code
https://07start.vercel.app/api/pds?action=bootstrap	GET	200 OK

로그인 상태의 동일 자료 요청 200 OK

Request URL	Request Method	Status Code
https://07start.vercel.app/api/pds?action=bootstrap	GET	401 Unauthorized

로그아웃 후 동일 자료 요청 401 Unauthorized

4. 남의 자료가 안 열리는 것을 보이기

검증 항목	확인 내용	결과
양방향 접근	rayeon→test · test→rayeon 자료 접근 모두 404 거절	PASS
수정·삭제 차단	다른 계정 ToDo 수정·삭제 요청은 NOT_FOUND	PASS
목록 격리	다른 계정 plan·todo·제목 미포함, 내 plan 만 포함	PASS
비로그인 접근	인증 없는 자료 요청은 401 AUTH_REQUIRED	PASS
소유권 기준	plan_id 와 현재 persona_id 를 함께 조건으로 조회	PASS
거절 소스 위치	api/pds.js read-plan 소유권 검사에서 404 반환	PASS

자료 소유권은 요청에서 받은 다른 계정 값이 아니라 로그인 세션의 persona_id 로 결정합니다. 다른 계정의 Plan-ToDo 를 읽거나 수정·삭제하려 하면 404 로 거절되고, 목록 응답에도 다른 계정 자료가 섞이지 않는지 확인했습니다. 거절된 요청은 반대편 자료에 반영되지 않아 기존 자료와 건수가 그대로 유지되는 것도 확인했습니다.

```

r = await fetch('/api/pds?action=delete-todo', {
  method: 'DELETE',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    todoid: 'todo-study-001-b'
  })
});
console.log('[삭제] STATUS:', r.status, 'RESPONSE:', await r.text());
})();

< Promise {<pending>}
  > GET https://07start.vercel.app/api/pds?action=read-plan&planid=plan-study-b-001 404 (Not Found) VM66:14
[읽기] STATUS: 404 RESPONSE: {"error": "Plan을 찾을 수 없어요.", "code": "NOT_FOUND"} VM66:5
  > PATCH https://07start.vercel.app/api/pds?action=update-todo 404 (Not Found) VM66:17
[수정] STATUS: 404 RESPONSE: {"error": "ToDo를 찾을 수 없어요.", "code": "NOT_FOUND"} VM66:15
  > DELETE https://07start.vercel.app/api/pds?action=delete-todo 404 (Not Found) VM66:17
[삭제] STATUS: 404 RESPONSE: {"error": "ToDo를 찾을 수 없어요.", "code": "NOT_FOUND"} VM66:24
  >

```

rayeon 에서 다른 계정 자료 읽기·수정·삭제 요청 404 거절

```

r = await fetch('/api/pds?action=delete-todo', {
  method: 'DELETE',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({
    todoid: 'todo-18103e5d-2e89-47d0-97c5-4212b54e3cc3'
  })
});
console.log('[삭제] STATUS:', r.status, 'RESPONSE:', await r.text());
})();

< Promise {<pending>}
  > GET https://07start.vercel.app/api/pds?action=read-plan&planid=plan-study-001 404 (Not Found) VM76:14
[읽기] STATUS: 404 RESPONSE: {"error": "Plan을 찾을 수 없어요.", "code": "NOT_FOUND"} VM76:5
  > PATCH https://07start.vercel.app/api/pds?action=update-todo 404 (Not Found) VM76:17
[수정] STATUS: 404 RESPONSE: {"error": "ToDo를 찾을 수 없어요.", "code": "NOT_FOUND"} VM76:15
  > DELETE https://07start.vercel.app/api/pds?action=delete-todo 404 (Not Found) VM76:17
[삭제] STATUS: 404 RESPONSE: {"error": "ToDo를 찾을 수 없어요.", "code": "NOT_FOUND"} VM76:24
  >

```

test 에서 반대 계정 자료 읽기·수정·삭제 요청 404 거절

```

fetch('/api/pds?action=bootstrap')
.then(async r => {
  const text = await r.text();
  console.log('STATUS:', r.status);
  console.log('다른 계정 plan 포함?', text.includes('plan-study-b-001'));
  console.log('다른 계정 todo 포함?', text.includes('todo-study-001-b'));
  console.log('다른 계정 제목 포함?', text.includes('내트위크 보안 실습 북스'));
  console.log('내 plan 포함?', text.includes('plan-study-001'));
});

< Promise {<pending>}
  STATUS: 200 VM110:4
  다른 계정 plan 포함? false VM110:5
  다른 계정 todo 포함? false VM110:6
  다른 계정 제목 포함? false VM110:7
  내 plan 포함? true VM110:8

```

목록 응답 확인 — 다른 계정 자료 false · 내 plan true

```

fetch('/api/pds?action=ownership-probe&personaId=user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab&username=test', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'X-User-Id': 'user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab',
    'X-Username': 'test'
  },
  body: JSON.stringify({
    personaId: 'user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab',
    personaId: 'user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab',
    username: 'test',
    planId: 'plan-study-b-001'
  })
}).then(async r => console.log('STATUS:', r.status, 'RESPONSE:', await r.text()));

< Promise {<pending>}
  STATUS: 200 RESPONSE: {"authenticatedUser": "rayeon", "serverOwner": {"personaId": "user-93ae32c7-5e65-40e6-83ef-f09f40bc332b", "planId": "plan-study-001"}, "ignoredSpoo": {"queryPersona": "user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab", "headerPersona": "user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab", "bodyPersona": "user-a0fc81d1-a374-4ea4-b395-d2d22bd9d7ab", "returnedPlanId": "plan-study-001", "returnedTodoId": "todo-18103e5d-2e89-47d0-97c5-4212b54e3cc3", "todo-b394c8b-cbbe-4d1b-b4d6-c5ce777055e8", "todo-bc702039-d104-4dab-b4f8-4a1c04c4e272", "todo-study-002", "todo-study-003"}

```

다른 계정 정보를 요청해도 로그인 세션의 rayeon 계정으로 처리

```

// T07 Card 4 evidence routes: ownership is always derived from the authenticated session.
// Supplying another account/plan in URL, headers, or body never changes currentPersona/currentPlan.
if (action === 'read-plan' && req.method === 'GET') {
  const requestedPlanId = String(req.query.planId || '').trim();
  if (!requestedPlanId) return bad(res, 'planId가 필요해요.~');
  if (!requestedPlanId) return bad(res, 'planId가 필요해요.~');
  const q = await client.query('SELECT plan_id, title, start_date, end_date, priority, success_criterion, estimated_minutes FROM plans WHERE plan_id=$1 AND persona_id=$2', [requestedPlanId, personaId]);
  if (q.rowCount) return bad(res, "Plan을 찾을 수 없어요.", 404, 'NOT_FOUND');
  const row = q.rows[0];
  return json(res, 200, {plan: {planId: row.plan_id, title: row.title, startDate: dateOnly(row.start_date), endDate: dateOnly(row.end_date), priority: row.priority, successCriterion: row.successCriterion, estimatedMinutes: row.estimated_minutes}});
}

```

소유권 검사 — plan_id 와 현재 persona_id 로 조회하고 없으면 404(Not Found) 거절

5. 설명서로 묶고, 잠금 앱으로 5 일 써 보기

앞의 인증 구현과 검증 결과를 여섯 항목으로 묶고, 잠긴 계정에서 5 일 사용 기록과 계획 규칙 변경 전후를 같은 지표로 비교했습니다.

① 무엇으로 붙였나

가입·로그인·로그아웃·세션·자료 소유권 검사는 api/pds.js 에서 직접 구현하고, 비밀번호 해시는 bcrypt(^6.0.0), 로그인 상태 유지는 t07_session HttpOnly 쿠키와 auth_sessions 서버 세션을 사용했습니다.

② 왜 그걸 골랐나

인증 흐름을 직접 설명할 수 있고 로그아웃 시 서버 세션을 즉시 끊을 수 있어 이 방식을 선택했습니다. Supabase Auth 와 JWT 도 검토했지만 외부 의존과 토큰 무효화 관리 부담 때문에 사용하지 않았습니다.

③ 어디를 어떻게 고쳤나

api/pds.js 에서 가입 시 bcrypt.hash(password, 12)로 비밀번호를 해시하고, 로그인 시 bcrypt.compare 로 확인한 뒤 서버 세션을 생성합니다. 로그아웃 시 서버 세션과 쿠키를 제거하며, 자료 조회는 plan_id 와 현재 로그인 세션의 persona_id 를 함께 조건으로 검사하도록 구현했습니다.

④ 안 열리는 것을 확인한 기록

정상 로그인과 로그인 상태의 자료 요청은 성공하고, 잘못된 로그인은 거절되는지 확인했습니다. 로그인 상태의 동일 자료 요청은 200, 로그아웃 뒤 같은 요청은 401, 다른 계정의 Plan·ToDo 읽기·수정·삭제는 404 로 거절되며 목록에는 현재 계정 자료만 포함되는 것도 확인했습니다.

⑤ AI 와 나

AI 에게는 인증 코드 구조와 카드별 통과 기준 대조, 검증 순서와 문서 구성을 맡겼습니다. 실제 사용할 계정과 기록, 규칙 변경 시점·이유, 제출에 사용할 캡처는 직접 확인하고 선택했으며, 실제로 확인하지 않은 항목은 PASS 로 적지 않았습니다.

⑥ 아직 못 막은 것

로그인 무차별 대입에 대한 시도 횟수 제한은 아직 구현하지 않았습니다. 반복적인 비밀번호 추측 요청을 제한하지 못하므로 계정 탈취 시도에 노출될 위험이 있습니다.

5-1. 5 일 사용 기록 · 규칙 변경 전후 비교

질문 계획한 학습시간을 줄이면 실제 계획 달성률이 높아지는가?

관찰 지표 · 단위 계획 달성률(%)

계산 규칙 실제 학습시간 ÷ 해당 날짜의 계획 기준시간 × 100, 소수 둘째 자리에서 반올림해 소수 첫째 자리까지 표시

계획 규칙 1~2 일차는 하루 150 분을 기준으로 하고, 2 일차 기록 뒤 3 일차 시작 전에 하루 120 분으로 한 번 변경

날짜	계획 기준	실제 / 변경 시각	달성률 / 변경 이유
2026-08-28	150 분	130 분	86.7%
2026-08-29	150 분	130 분	86.7%
규칙 변경	150 → 120 분	2026-08-30 09:15:27 KST	1~2 일차 기록 기준으로 조정
2026-08-30	120 분	120 분	100.0%
2026-08-31	120 분	130 분	108.3%
2026-09-01	120 분	130 분	108.3%

규칙 변경 시각 2026-08-30 09:15:27 KST · 변경 이유 1~2 일차 기록을 기준으로 3 일차부터 학습시간을 조정

5 일 계산 실제 130 + 130 + 120 + 130 + 130 = 640 분. 최종 예상 합계 660 분, 실제 640 분, 차이 -20 분으로 SEE 화면일치

계산 처리 기준

빠진 값은 임의로 보정하지 않고 누락으로 확인합니다. 중복 기록은 한 번만 계산합니다. 유난히 큰 값은 원본 기록을 다시 확인하고 실제 기록이면 그대로 유지합니다. 달성률은 소수 둘째 자리에서 반올림해 소수 첫째 자리까지 표시하며, 주 시작 요일은 월요일로 정했습니다.

내보내기·계정 삭제 내 자료 전체를 파일 하나로 내보낼 수 있는지 확인하고, 계정 삭제 시 내 자료도 함께 삭제된다는 안내를 확인합니다. 비밀번호·세션값·토큰·비밀키 원문은 문서와 캡처에 남기지 않습니다.

PLAN · DO · SEE DIARY 2Asia/Seoul · 로그인한 내 자료만 표시rayeonrecode로그아웃

PLANDOSEEDATA

계획5건

완료5건

지연0건

미완료0건

예상660분

실제640분

차이-20분

SEE REVIEW

계획과 실제 비교KST 검토 기준일 2026-09-02 · 차이 = 실제 - 예상

근거 보기: 전체 계획 · 집계 숫자를 누르면 해당 기록만 확인할 수 있습니다.

원본 ToDo	마감일	상태	예상	실제	판정	미완 이유 원본
실기 이론 강의 수강	2026-08-30	완료	220분	195분	완료	없음
실기 이론 핵심 개념 정리	2026-08-31	완료	70분	65분	완료	없음
프로그래밍 강의 수강	2026-09-01	완료	180분	150분	완료	없음
프로그래밍 문제 풀이	2026-09-01	완료	140분	170분	완료	없음
틀린 문제 및 부족한 내용 복습	2026-09-01	완료	50분	60분	완료	없음

SEE 최종 확인 — 예상 660 분 · 실제 640 분 · 차이 -20 분 · 완료 5 건 · 지연 0 건

6. 제출 전 최종 점검

스스로 점검 항목	확인
직접 구현 인증 방식과 bcrypt ^6.0.0 사용 이유, 검토했지만 선택하지 않은 방법과 이유를 기록했다.	✓
새 계정 가입·로그인·로그아웃이 동작하고 중복 아이디 가입은 거절된다.	✓
없는 아이디와 틀린 비밀번호에 동일한 로그인 실패 문구가 표시된다.	✓
로그인하지 않은 상태에서 자료 주소에 직접 접근하면 다이어리 대신 로그인 화면이 표시된다.	✓
비밀번호는 bcrypt 해시로 저장되고 같은 비밀번호를 사용한 계정도 서로 다른 해시값으로 저장된다.	✓
로그인 요청·응답과 제출 화면에 비밀번호 원문 또는 비밀번호 해시값이 노출되지 않는다.	✓
로그인 상태는 t07_session HttpOnly 쿠키와 auth_sessions 서버 세션으로 유지되고 6 시간 만료 기준이 적용된다.	✓
인증값은 URL 에 넣지 않고 HttpOnly · Secure · SameSite=Strict 쿠키 보호 속성을 확인했다.	✓
로그아웃 후 같은 자료 요청이 401 로 거절되고, 비밀번호 변경 시 기존 세션도 더 이상 사용할 수 없다.	✓
두 실제 계정에서 서로의 Plan·ToDo 읽기·수정·삭제를 시도했을 때 양방향 모두 거절된다.	✓
주소·헤더·본문에 다른 계정 정보를 넣어도 현재 로그인 세션의 계정 소유권 기준이 유지된다.	✓
목록 응답에 다른 계정 자료가 섞이지 않고, 거절 전후 반대편 자료와 건수가 그대로 유지된다.	✓
5 개의 서로 다른 실제 날짜 기록을 같은 질문·지표·단위·계산 규칙으로 작성했다.	✓
2 일차 기록 뒤 3 일차 시작 전에 계획 기준을 150 분에서 120 분으로 한 번 변경하고 시각과 이유를 기록했다.	✓
누락·중복·이상값·반올림 처리 기준과 주 시작 요일을 정하고 5 일 합계·평균을 수동 계산과 대조했다.	✓
내 자료 전체를 한 파일로 내보낼 수 있는지 확인하고 계정 삭제 시 내 자료도 함께 삭제된다는 안내를 확인했다.	✓
검증 안내서 4 개 항목, AI 3 줄, 인증 구현 설명 6 개 항목을 작성했다.	✓
제출 문서와 캡처에 비밀번호·세션값·토큰·비밀키 원문을 남기지 않았다.	✓
과제 07 이 이어받은 과제 06 최종 결과와 기준 커밋을 명시하고, 과제 07 소스 이력에서 그 기준을 이어받았다.	✓

제출용 요약

결과물 주소	https://07end.vercel.app/
소스 주소	https://github.com/rykim1217/ALEPH/tree/main/submit/task07
확인 4 줄	① 가입·로그인·로그아웃 및 비로그인 차단 ② bcrypt 비밀번호 보호·서버 세션 ③ 계정별 자료 소유권 격리 ④ 5 일 기록·규칙 변경·내보내기·계정 삭제
공개 범위	첫 공개 결과 페이지는 로그인 화면 · 로그인한 계정의 기록은 공개되지 않음
과제 06 기준	최종 기준 커밋 ae4cf4cdafc6842b45baae90ae79be96bd584336